

Three.js : Lumières

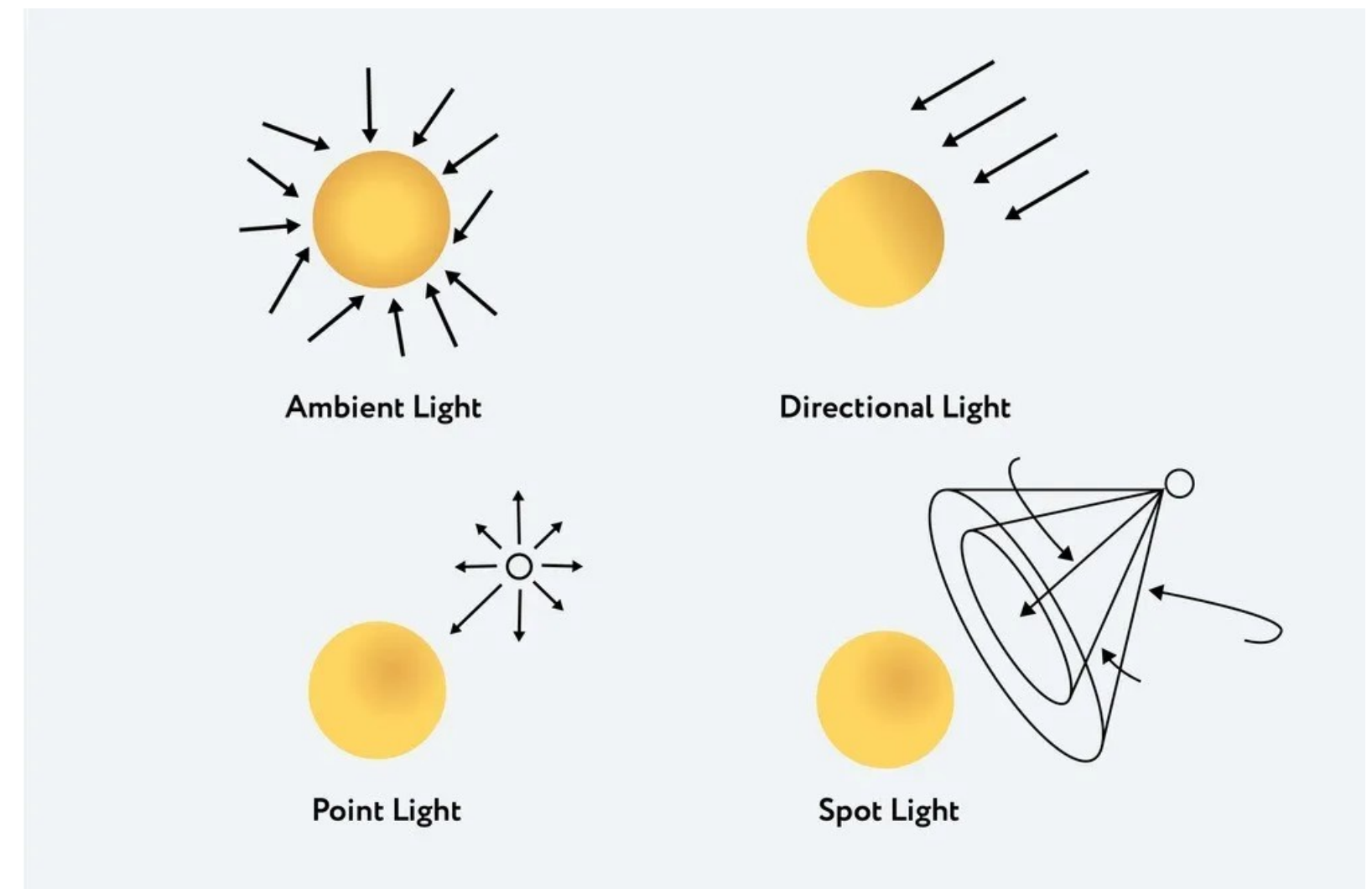


Photo réalisme



"90% of the time, the materials and lighting carry the weight of creating truly photorealism imagery"

Alex Roman, From Bits to Lens

Ambient Light

```
const light = new THREE.AmbientLight( 0x404040 ); // soft white light
scene.add( light );
```

AmbientLight(color : Integer, intensity : Float)

color - (optional) Numeric value of the RGB component of the color. Default is 0xffffff.

intensity - (optional) Numeric value of the light's strength/intensity. Default is 1.

Lumière qui affecte la scene de manière globale, sans gestion des ombres ou des réflexions

Hemisphere Light

HemisphereLight(skyColor : Integer, groundColor : Integer, intensity : Float)

skyColor - (optional) hexadecimal color of the sky. Default is 0xffffffff.

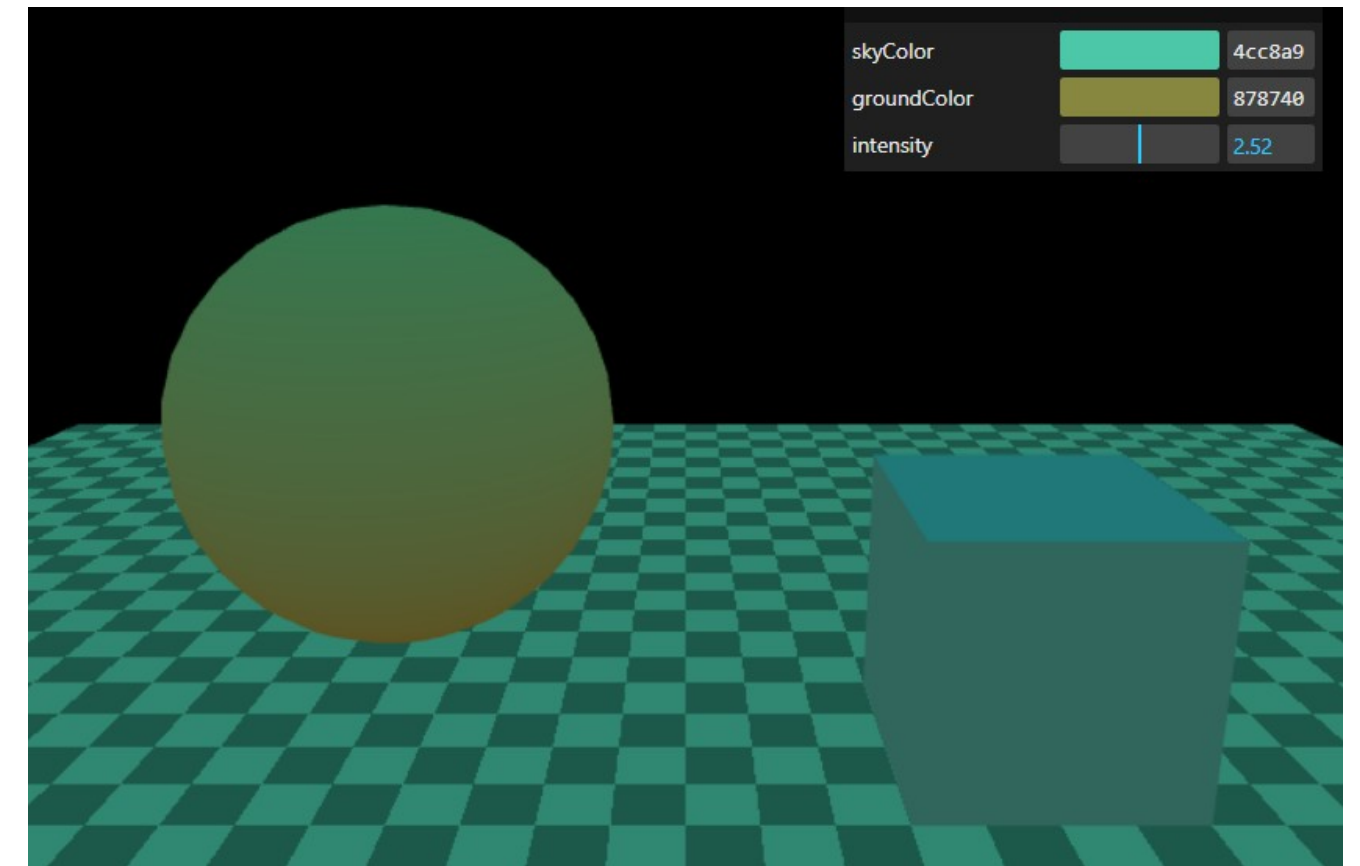
groundColor - (optional) hexadecimal color of the ground. Default is 0xffffffff.

intensity - (optional) numeric value of the light's strength/intensity. Default is **1**.

Creates a new HemisphereLight.

Source de lumière qui donne un effet de dégradé entre le haut et le bas de la scene entre 2 couleurs.

L'HemisphereLight n'affecte pas les ombres mais peut faire briller les matériaux PBR



Directional Light

`DirectionalLight( color : Integer, intensity : Float )`

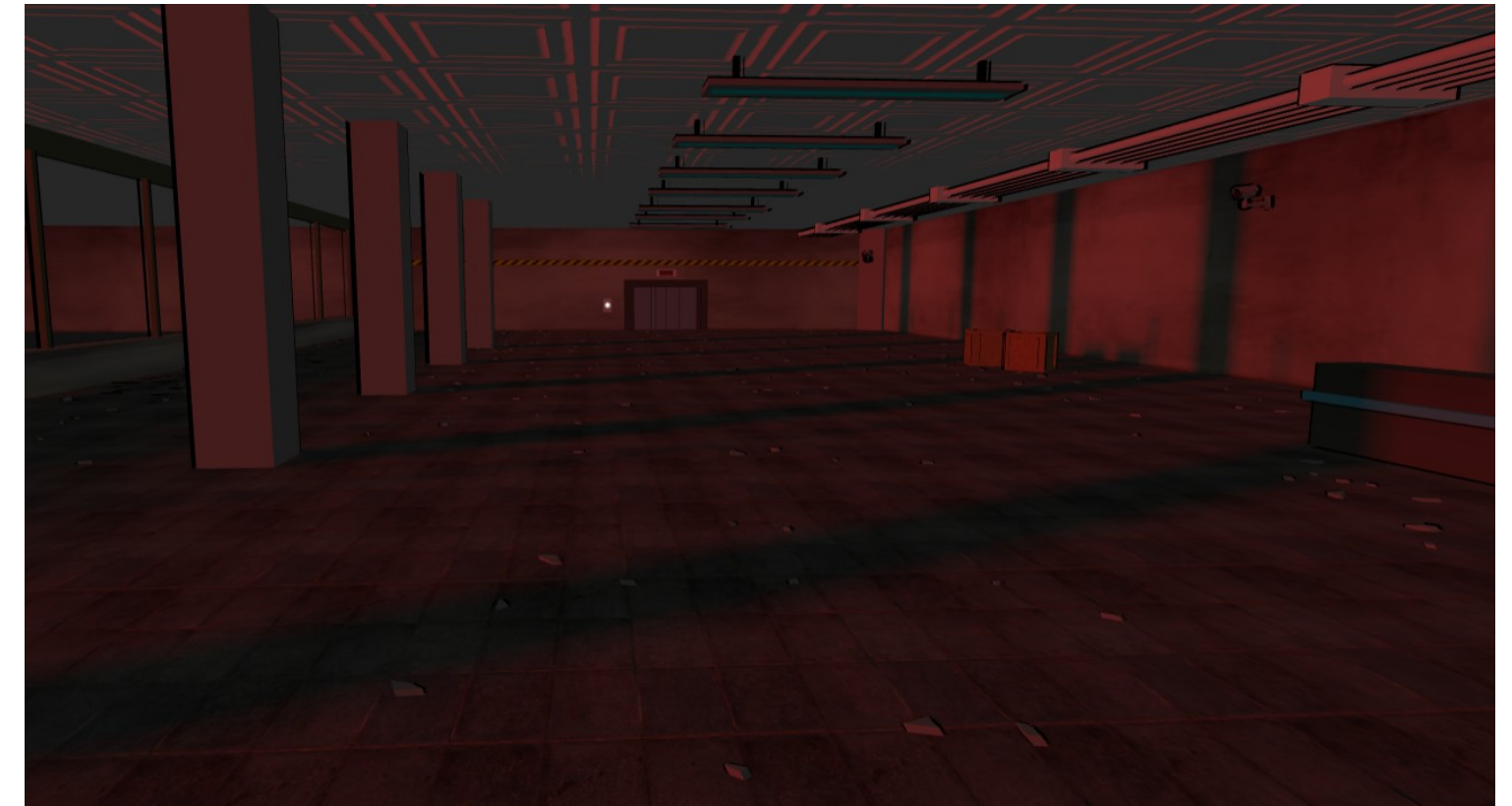
`color` - (optional) hexadecimal color of the light. Default is 0xffffffff (white).

`intensity` - (optional) numeric value of the light's strength/intensity. Default is 1.

Source qui émet de la lumière dans une direction

La direction est définie par la position de la DirectionalLight et son orientation

Une DirectionalLight peut projeter des ombres et affecte les réflexions



Point Light

`PointLight( color : Integer, intensity : Float, distance : Number, decay : Float )`

`color` - (optional) hexadecimal color of the light. Default is 0xffffffff (white).

`intensity` - (optional) numeric value of the light's strength/intensity. Default is `1`.

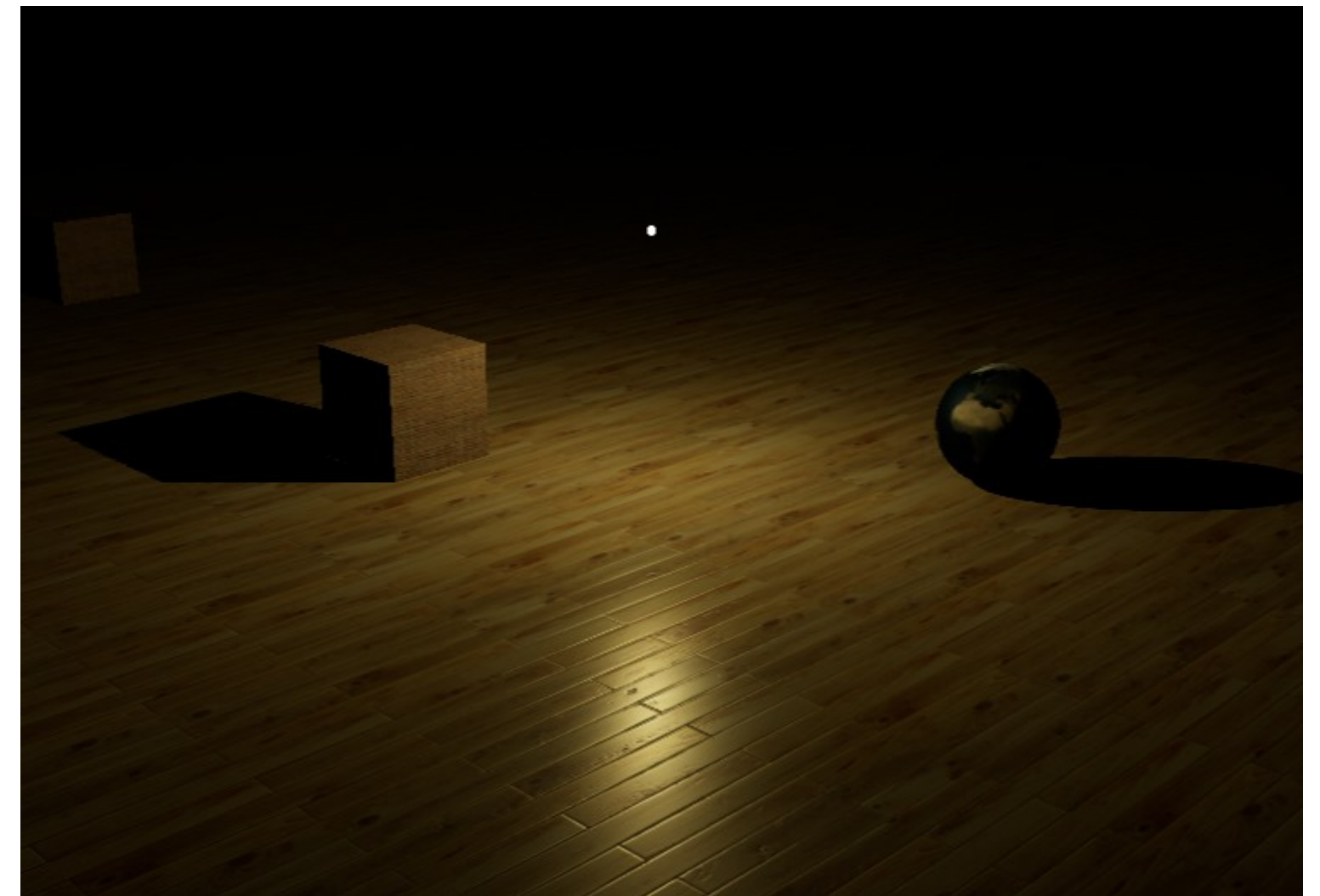
`distance` - Maximum range of the light. Default is `0` (no limit).

`decay` - The amount the light dims along the distance of the light. Default is `2`.

Creates a new PointLight.

Source qui émet de lumière dans toutes les directions à partir d'un point donné

Une PointLight peut projeter des ombres et affecte les réflexions



Spot Light

SpotLight(color : Integer, intensity : Float, distance : Float, angle : Radians, penumbra : Float, decay : Float)

color - (optional) hexadecimal color of the light. Default is 0xffffffff (white).

intensity - (optional) numeric value of the light's strength/intensity. Default is **1**.

distance - Maximum range of the light. Default is **0** (no limit).

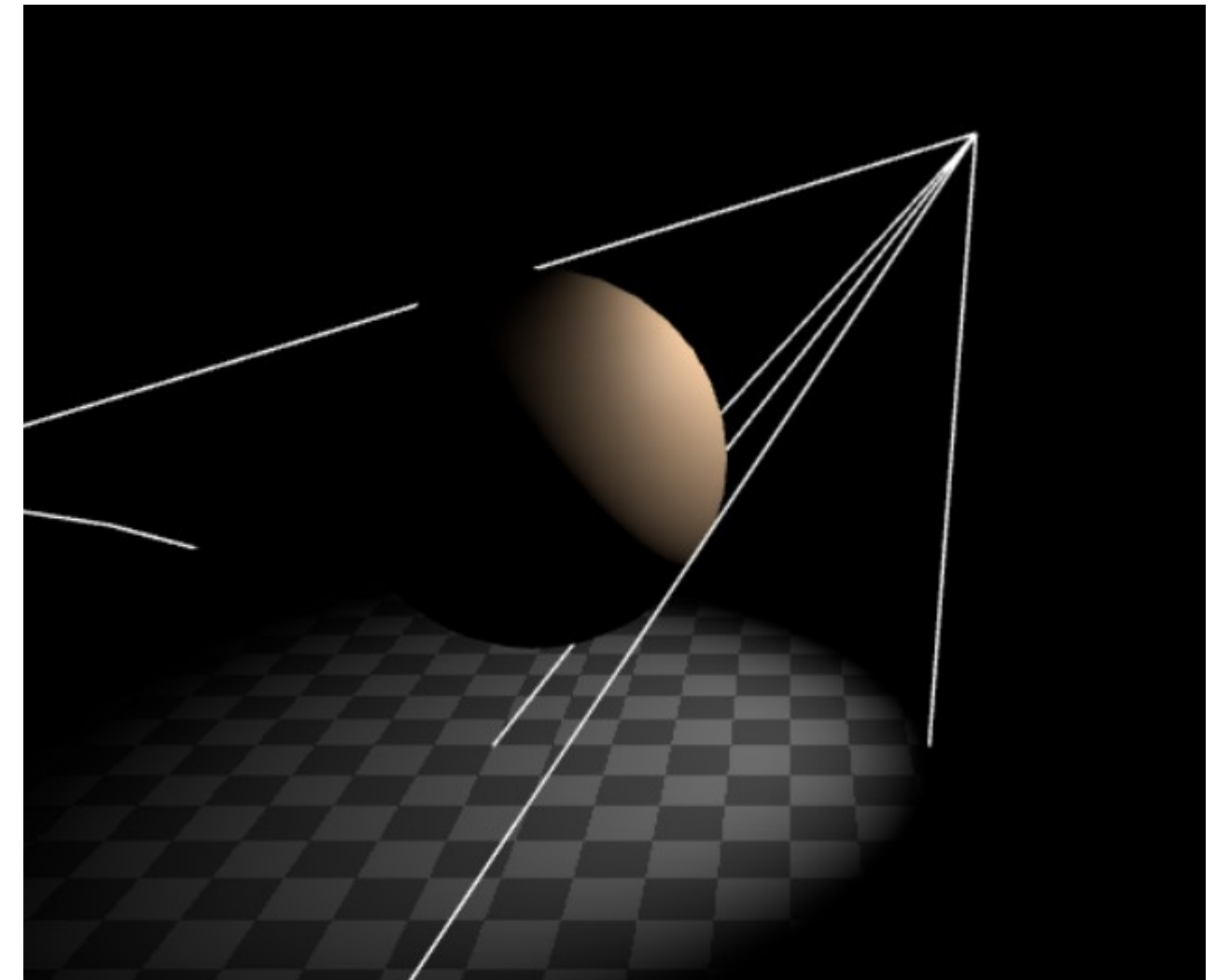
angle - Maximum angle of light dispersion from its direction whose upper bound is $\text{Math.PI}/2$.

penumbra - Percent of the spotlight cone that is attenuated due to penumbra. Takes values between zero and **1**. Default is zero.

decay - The amount the light dims along the distance of the light.

Source qui émet de lumière dans un cone partir d'un point donné

Une SpotLight peut projeter des ombres et affecte les réflexions



Rect Area Light

`RectAreaLight( color : Integer, intensity : Float, width : Float, height : Float )`

`color` - (optional) hexadecimal color of the light. Default is 0xffffffff (white).

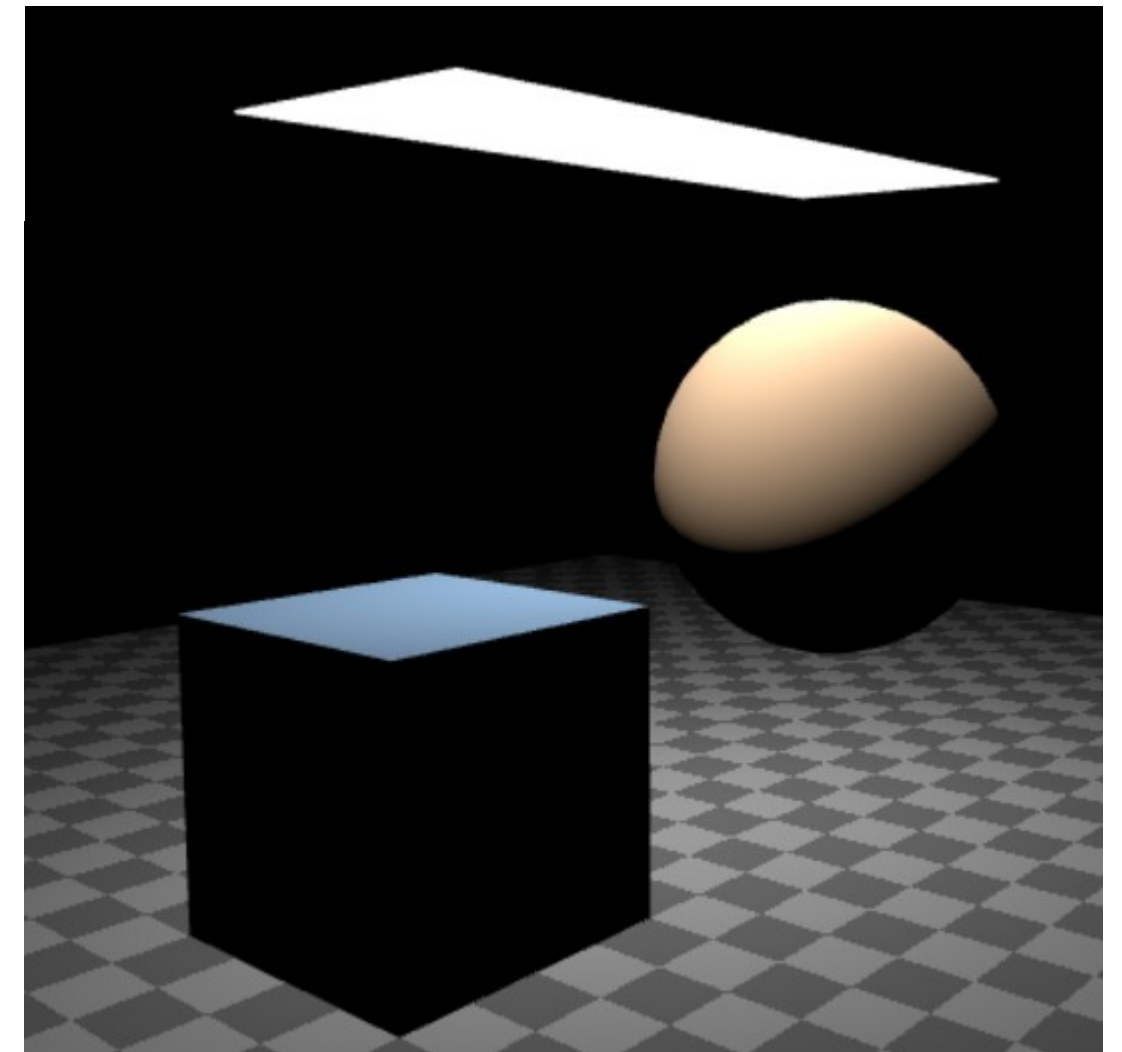
`intensity` - (optional) the light's intensity, or brightness. Default is `1`.

`width` - (optional) width of the light. Default is `10`.

`height` - (optional) height of the light. Default is `10`.

Source qui émet de lumière uniformément depuis une zone rectangulaire

Une RectAreaLight n'affecte pas les ombres mais peut faire briller les matériaux PBR

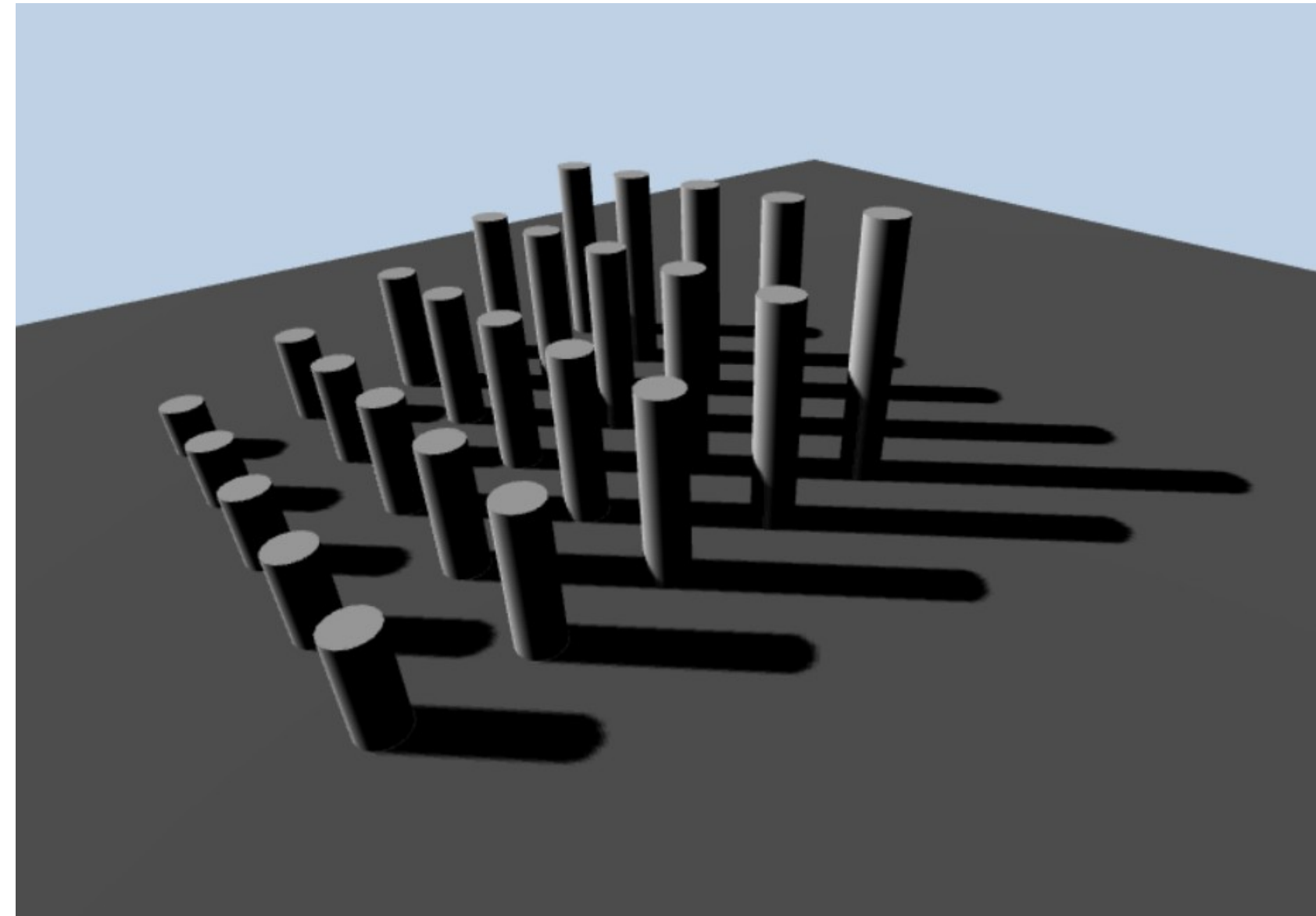


Gestion des ombres

Ombres

Pour avoir des ombres sur les scenes, il faut configurer les choses suivantes :

- Utiliser des lumières qui influencent les ombres
- Définir les Mesh projettent des ombres
- Définir les Mesh qui reçoivent les ombres



Ombre avec une Directional Light

Activer les ombres

```
var light = new THREE.DirectionalLight(0xffffff, 1);  
light.castShadow = true;
```

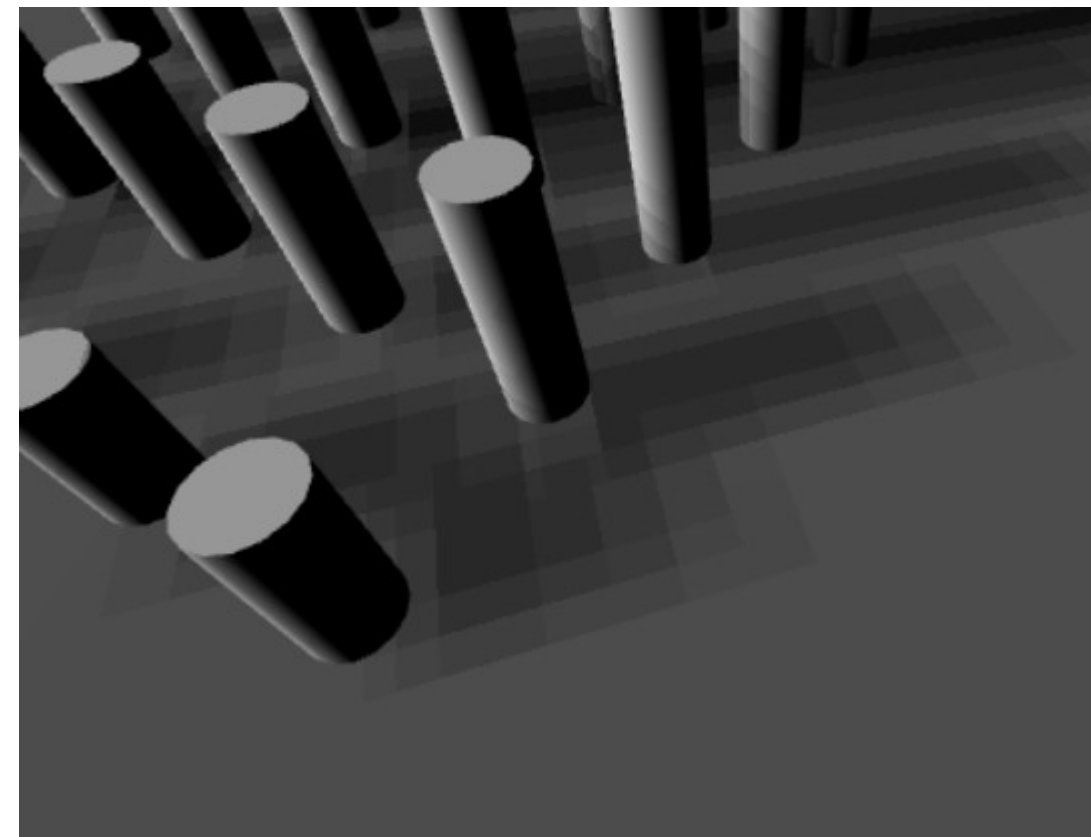
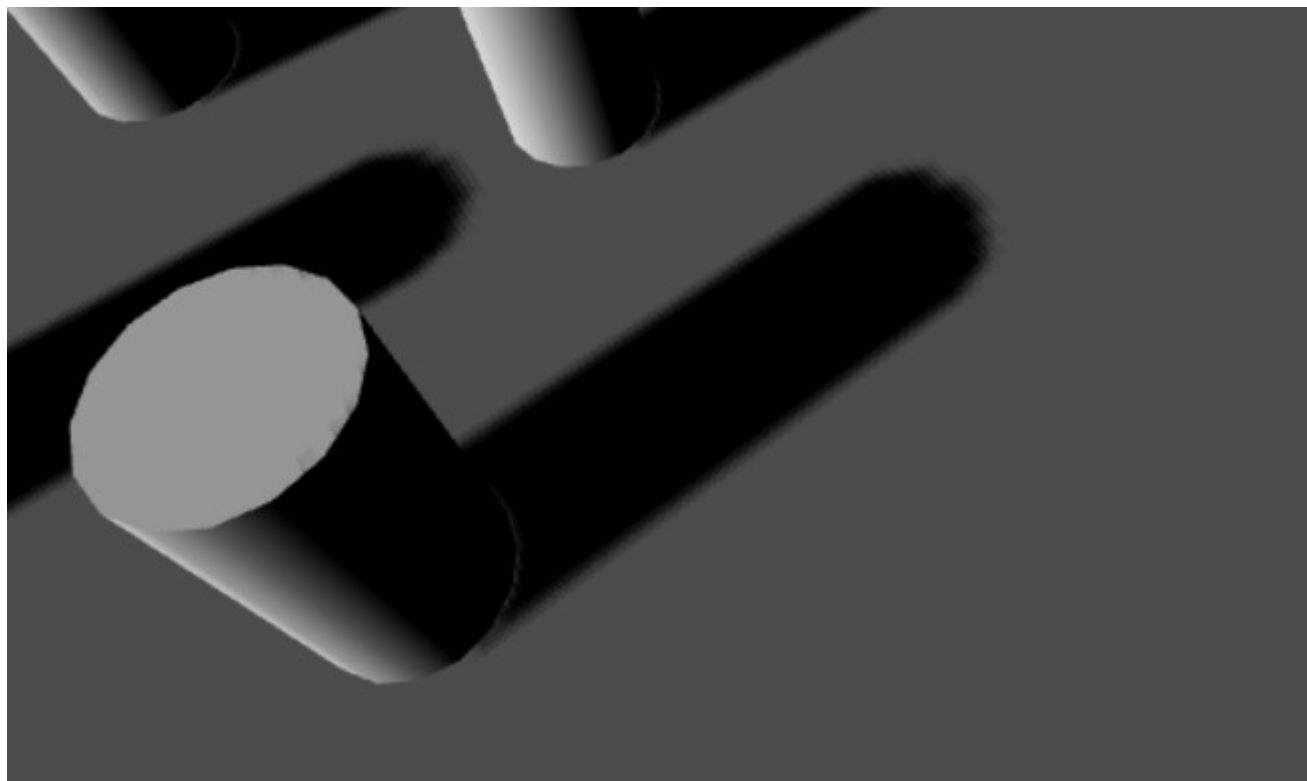
Définir la distance sur laquelle calculer les ombres

```
light.shadow.camera.left = -100;  
light.shadow.camera.right = 100;  
light.shadow.camera.top = 100;  
light.shadow.camera.bottom = -100;  
light.shadow.camera.near = 1;  
light.shadow.camera.far = 200;
```

Ombre avec une Directional Light

Définir la résolution des ombres

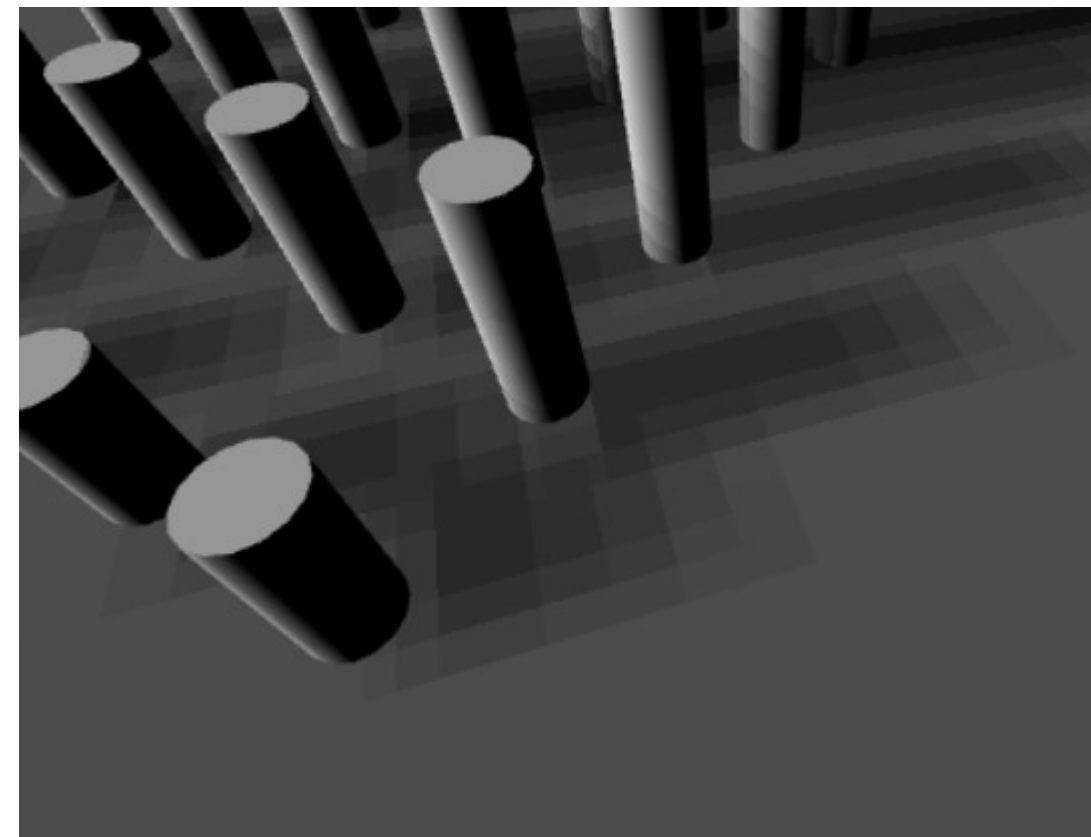
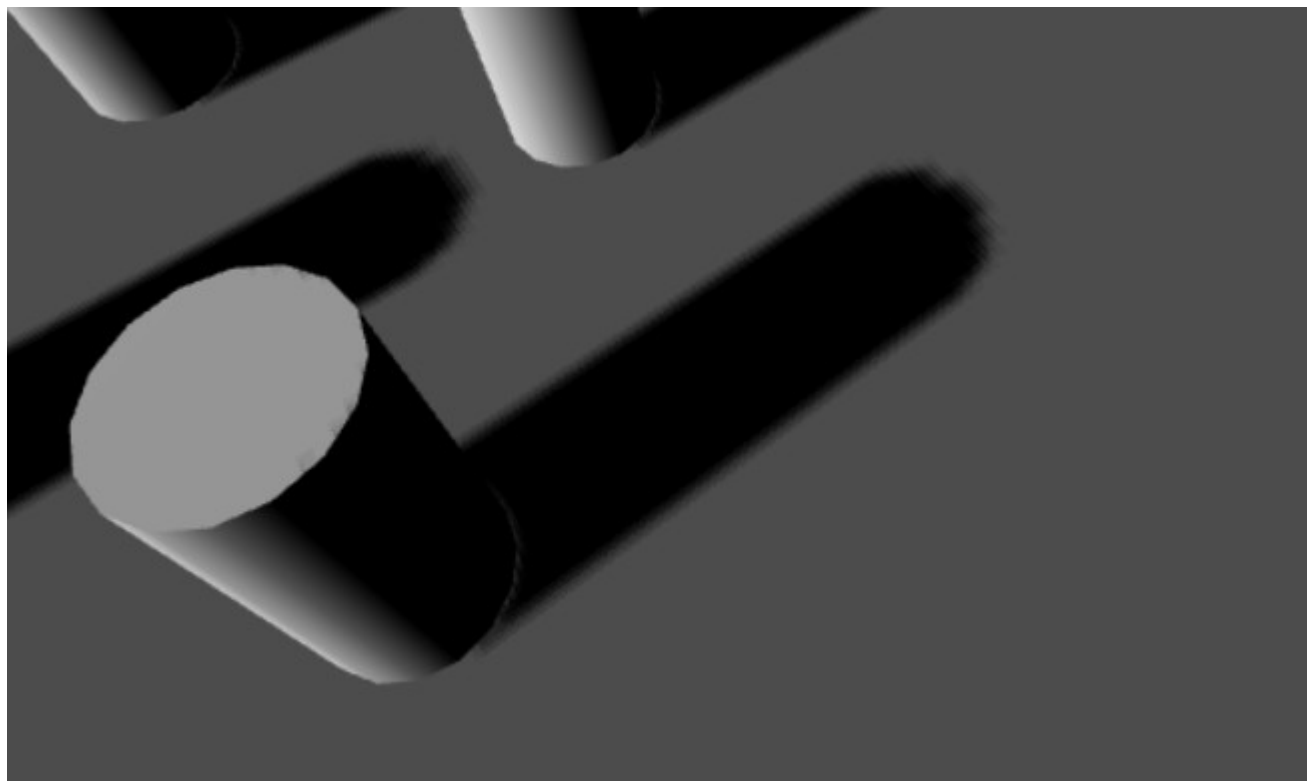
```
light.shadow.mapSize.width = 1024;  
light.shadow.mapSize.height = 1024;
```



Ombre avec une Directional Light

Définir la résolution des ombres

```
light.shadow.mapSize.width = 1024;  
light.shadow.mapSize.height = 1024;
```



Ombre avec une Directional Light

Il faut ensuite définir sur chaque mesh, si ils reçoivent ou projettent des ombres

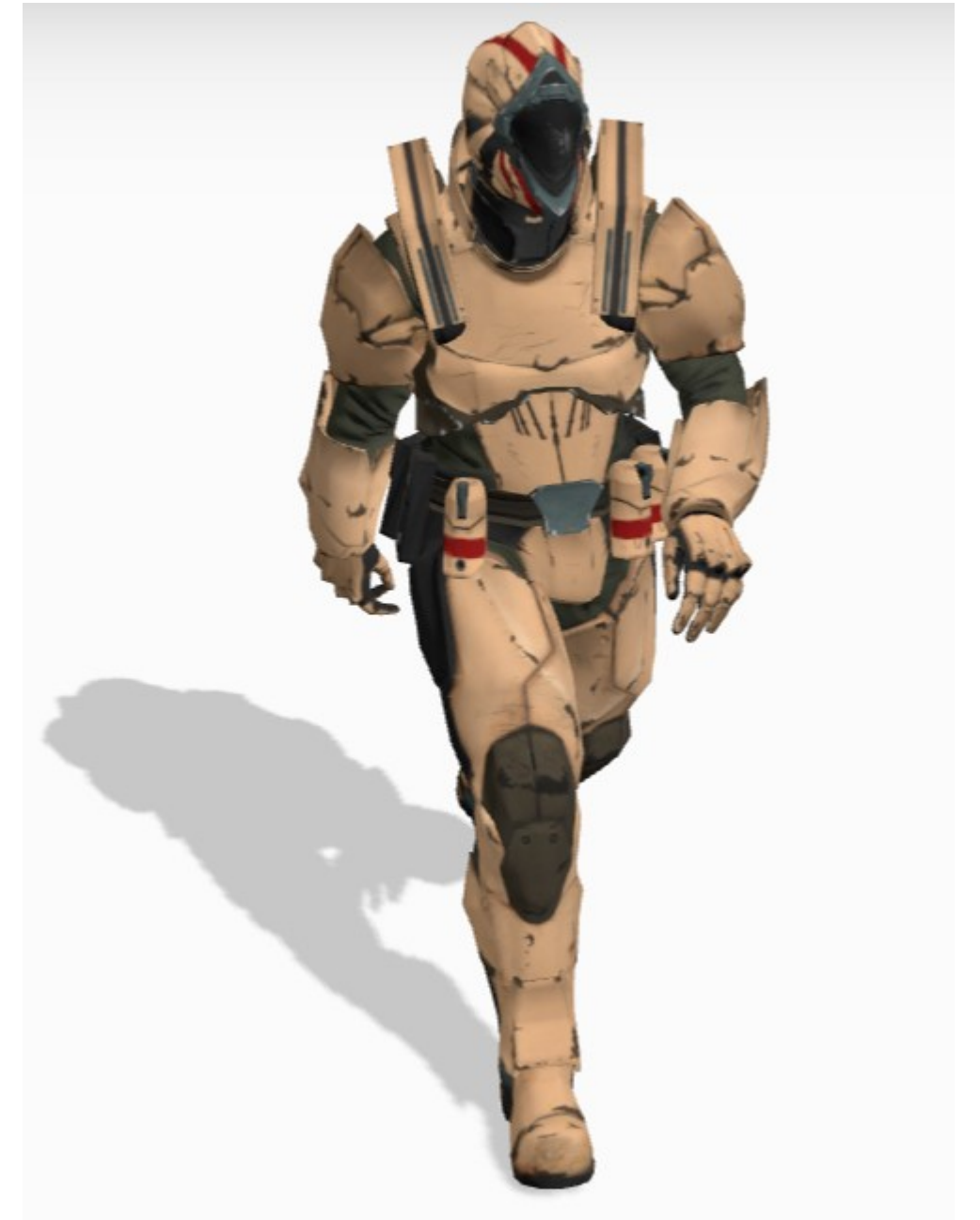
```
mesh.castShadow = true;  
mesh.receiveShadow = true;
```

Charger un modèle 3D

Modèle 3d

Three.js peut charger des modèles 3D de différents formats (glb, gltf, fbx, obj ...)

Pour le web il est recommandé d'utiliser le format glb car il est plus compact et peut contenir les animations et les textures des modèles



Charger un modèle glb/gltf

Pour charger un modèle, il faut utiliser la classe Loader associé au bon format

```
import { GLTFLoader } from 'https://cdn.skypack.dev/  
three@0.132.2/examples/jsm/loaders/GLTFLoader.js';
```

```
var loader = new GLTFLoader();  
loader.load( '../Soldier.glb', function ( gltf ) {  
  
    model = gltf.scene;  
    scene.add( model );  
  
});
```


Charger un modèle glb/gltf

La scene représente un groupe entier contenant tout les modèles de la scene

En utilisant la propriété **children** des groupes ou en utilisant la méthode **traverse**, il est possible de parcourir les Mesh enfant individuellement

```
for( let child of model.children )  
{  
    //...  
}
```

```
model.traverse(child => {  
    scene.add( child );  
});
```

Gestion des animations

Les modèles glb peuvent contenir des animations. Pour pouvoir les jouer il faut utiliser la classe **AnimationMixer**

```
mixer = new THREE.AnimationMixer( model );
```

Il faut ensuite aller récupérer l'animationClip de l'animation à jouer sur le modèle et appeler la méthode **play**

```
var animations = gltf.animations;  
  
var animation = mixer.clipAction( animations[ 0 ] );  
animation.play();
```

Exercices